Thomas Nguyen
1188002
CPTS 360
PA 3
Introduction to Linux Kernel Programming

Implementation

For this assignment, I implemented a Linux kernel module in C that monitors the CPU usage of registered processes. The module allows userspace applications to register their process IDs (PIDs) through the /proc filesystem. It stores the registered processes and their CPU times in a linked list and updates the values every 5 seconds using a kernel timer and workqueue.

The module creates a directory /proc/kmlab and a file /proc/kmlab/status when it is loaded. The status file has permissions 0666, allowing users to read and write to it. When the module is unloaded, these entries are removed and the allocated memory is freed.

Process Registration and Linked List

I used the Linux kernel's linked list implementation to store information about each registered process. Each process record contains a PID, its CPU time, and a list_head structure that connects it to the other records.

When a process writes its PID to /proc/kmlab/status, the write callback uses copy_from_user() to safely copy the input into kernel memory. The PID is then validated before a new process record is allocated using kmalloc() and added to the linked list. Duplicate PIDs are rejected.

The read callback allows applications to view the registered processes and their CPU times. It goes through the linked list and formats the information as a list of PIDs and CPU-time values. The results are then copied back to userspace.

Timer, Workqueue, and Locking

I implemented a kernel timer that runs every 5 seconds to schedule a workqueue. The timer schedules the work, while the workqueue handles updating the CPU times of registered processes.

The workqueue goes through the linked list and uses the provided get_cpu_use() function to retrieve each process's CPU time. If a process has terminated, it is removed from the linked list and its allocated memory is freed.

Since multiple processes can access the module, I used spinlocks to protect the linked list from race conditions. This prevents different parts of the module from modifying the list at the same time.

When the module is unloaded, the timer and scheduled work are stopped, the /proc entries are removed, and the remaining process records are freed.

Userspace Application

The userspace application uses getpid() to obtain its PID and registers itself by writing to /proc/kmlab/status. It then performs calculations for approximately 10–15 seconds to generate CPU activity.

After completing the calculations, the application reads /proc/kmlab/status and prints the registered processes along with their CPU times.

Testing

I compiled the kernel module and userspace application on Ubuntu Linux using the provided Makefile. Both compiled successfully without compiler warnings.

I tested the implementation by loading the module using insmod and verifying that /proc/kmlab/status was created with the correct permissions.

I then tested the userspace application and confirmed that it successfully registered its PID and displayed its CPU time.

I also ran the provided kmlab_test.sh script to test multiple processes. The output showed two registered PIDs, and the CPU-time values increased during later readings. The results also showed that terminated processes were removed from the list.

```
(base) vboxuser@UbuntuVM:~/WSU-CPTS/PAs/PA_3/PA_3/helper_files$ sudo insmod kmlab.ko
(base) vboxuser@UbuntuVM:~/WSU-CPTS/PAs/PA_3/PA_3/helper_files$ ./userapp
13296: 3544000000
```

Figure 1. CPU time reported for a single registered process.

```
(base) vboxuser@UbuntuVM:~/WSU-CPTS/PAs/PA_3/PA_3/helper_files$ bash kmlab_test.sh
make -C /lib/modules/5.15.0-139-generic/build M=/home/vboxuser/WSU-CPTS/PAs/PA_3/PA_3/helper_files modules
make[1]: Entering directory '/usr/src/linux-headers-5.15.0-139-generic'
make[1]: Leaving directory '/usr/src/linux-headers-5.15.0-139-generic'
== read 1 ==
13747: 0
13747: 0
13746: 0
13746: 0
== read 2 ==
13747: 936000000
== read 3 ==
13747: 6576000000
```

Figure 2. CPU-time monitoring with two registered processes.